

PEC4: Introducción al aprendizaje computacional

Presentación

Cuarta PEC del curso de Inteligencia Artificial

Competencias

En esta PEC se trabajarán las siguientes competencias:

Competencias de grado:

- Capacidad de analizar un problema con el nivel de abstracción adecuado a cada situación y aplicar las habilidades y conocimientos adquiridos para abordarlo y solucionarlo.

Competencias específicas:

- Conocer los diferentes aspectos del aprendizaje computacional (aprendizaje supervisado, no supervisado y por refuerzo).
- Conocer los fundamentos teóricos de los métodos más representativos.
- Conocer el tratamiento de los conjuntos de datos para la correcta validación de los sistemas de aprendizaje.

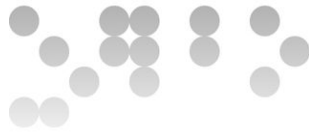
Objetivos

Esta PEC pretende evaluar diferentes aspectos de aprendizaje supervisado y no supervisado.

Descripción de la PEC a realizar

Pregunta 1)

Un sistema de estabilización de aviones necesita encenderse o apagarse según dos sensores x e y . Estos sensores tienen valores reales que van de $-\infty$ a ∞ , y el control se puede encender (*on*) o apagar (*off*). Debido al alto coste de acceder a los sensores, solo se han obtenido 16 instancias con la predicción de control y los valores de los sensores. El objetivo del sistema es entrenar un algoritmo para todos los valores de x e y .



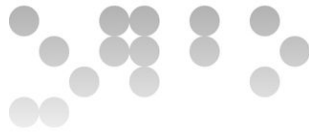
Inst	Clase	X	Y
1	off	-1	2
2	off	-2	2
3	on	2	2
4	on	1	2
5	on	-3	-1
6	on	-5	-1
7	off	4	-1
8	off	2	-1
9	off	-2	1
10	off	-4	1
11	on	4	4
12	on	6	7
13	on	-9	-9
14	on	-2	-1
15	off	3	-3
16	on	-2	1

Apartado 1 (puntos 3)

Primero de todo construiremos un árbol de decisión con las características x e y . Como dichas características son números reales, debemos de aplicar una función para binarizar x e y . De esta forma podremos calcular la bondad y decidir la mejor binarización para separar las instancias. Por lo tanto, calcular la binarización de los datos con las siguientes funciones y construir un árbol de decisión a partir de la bondad.

$$\begin{cases} 0, \text{if } x \leq -1 \\ 1, \text{if } x > -1 \end{cases} \quad \begin{cases} 0, \text{if } y \leq 0 \\ 1, \text{if } y > 0 \end{cases}$$

$$\begin{cases} 0, \text{if } x \leq 0 \\ 1, \text{if } x > 0 \end{cases} \quad \begin{cases} 0, \text{if } y \leq 1 \\ 1, \text{if } y > 1 \end{cases}$$



Solución:

Primero de todo, se calculan 4 nuevos atributos, uno para cada una de las funciones de entrada. Una vez se calculen los nuevos atributos se puede calcular la bondad para decidir los mejores atributos para separar las instancias.

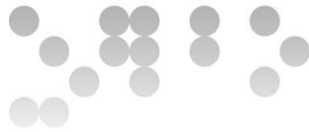
$$X0 \quad \begin{cases} 0, if \ x \leq -1 \\ 1, if \ x > -1 \end{cases}$$

$$X1 \quad \begin{cases} 0, if \ x \leq 0 \\ 1, if \ x > 0 \end{cases}$$

$$Y0 \quad \begin{cases} 0, if \ y \leq 0 \\ 1, if \ y > 0 \end{cases}$$

$$Y1 \quad \begin{cases} 0, if \ y \leq 1 \\ 1, if \ y > 1 \end{cases}$$

Inst	Clase	X	Y	X0	X1	Y0	Y1
1	off	-1	2	0	0	1	1
2	off	-2	2	0	0	1	1
3	on	2	2	1	1	1	1
4	on	1	2	1	1	1	1
5	on	-3	-1	0	0	0	0
6	on	-5	-1	0	0	0	0
7	off	4	-1	1	1	0	0
8	off	2	-1	1	1	0	0
9	off	-2	1	0	0	1	0
10	off	-4	1	0	0	1	0
11	on	4	4	1	1	1	1
12	on	6	7	1	1	1	1
13	on	-9	-9	0	0	0	0
14	on	-2	-1	0	0	0	0
15	off	3	-3	1	1	0	0



16	on	-2	1	0	0	1	0
----	----	----	---	---	---	---	---

Cálculo Bondad

X0:

- X0=1 off 3 on 4
- X0=0 off 4 on 5
- Bondad: $(5+4)/16 = 9/16$

X1:

- X1=1 off 3 on 4
- X1=0 off 4 on 5
- Bondad: $(5+4)/16 = 9/16$

Y0:

- Y0=1 off 4 on 5
- Y0=0 off 3 on 4
- Bondad: $(5+4)/16 = 9/16$

Y1:

- Y1=1 off 2 on 4
- Y1=0 off 5 on 5
- Bondad: $(5+4)/16 = 9/16$

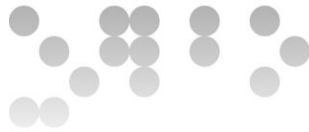
Al tener todos las características la misma bondad se puede usar cualquiera para dividir los datos.

Cogemos X0 para dividir los datos aunque se podría dividir los datos con X1, Y0 o Y1.

Nodo Raiz X0:

- X0 ≤ -1

Inst	Clase	X	Y	X1	Y0	Y1
1	off	-1	2	0	1	1
2	off	-2	2	0	1	1
5	on	-3	-1	0	0	0
6	on	-5	-1	0	0	0
9	off	-2	1	0	1	0
10	off	-4	1	0	1	0



13	on	-9	-9	0	0	0
14	on	-2	-1	0	0	0
16	on	-2	1	0	1	0

Ahora calcularemos la bondad con los tres atributos que faltan:

X1:

- X1=1 off 0 **on 0**
- X1=0 off 4 **on 5**
- Bondad: $(5+0)/9 = 5/9$

Y0:

- Y0=1 off 4 **on 1**
- Y0=0 off 0 **on 4**
- Bondad: $(4+4)/9 = 8/9$

Y1:

- Y1=1 **off 2** on 0
- Y1=0 off 2 **on 5**
- Bondad: $(5+2)/9 = 7/9$

Las instancias restantes se dividen con Y0. Siendo la salida de las hojas la siguiente:

Y0=0: Prob(on) = 1 y Prob(off)=0

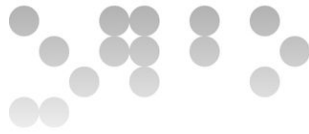
Y0=1: Prob(on) = 0.2 y Prob(off)=0.8

- **X0>-1**

Inst	Clase	X	Y	X1	Y0	Y1
3	on	2	2	1	1	1
4	on	1	2	1	1	1
7	off	4	-1	1	0	0
8	off	2	-1	1	0	0
11	on	4	4	1	1	1
12	on	6	7	1	1	1
15	off	3	-3	1	0	0

X1:

- X1=1 off 3 **on 4**



- $X1=0$ off 0 on 0
- Bondad: $(4+0)/7 = 4/7$

Y0:

- $Y0=1$ off 0 on 4
- $Y0=0$ off 3 on 0
- Bondad: $(4+3)/7 = 1$

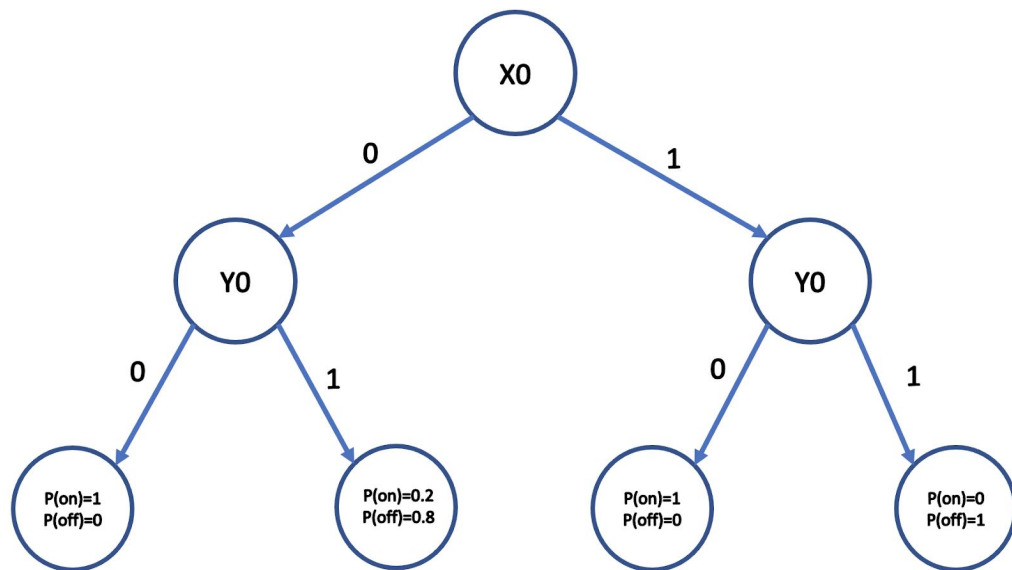
Y1:

- $Y1=1$ off 0 on 4
- $Y1=0$ off 3 on 0
- Bondad: $(5+2)/7 = 7/7$

Las instancias restantes se pueden dividir con Y0 o Y1, escogiendo Y0 como referencia. Siendo la salida de las hojas la siguiente:

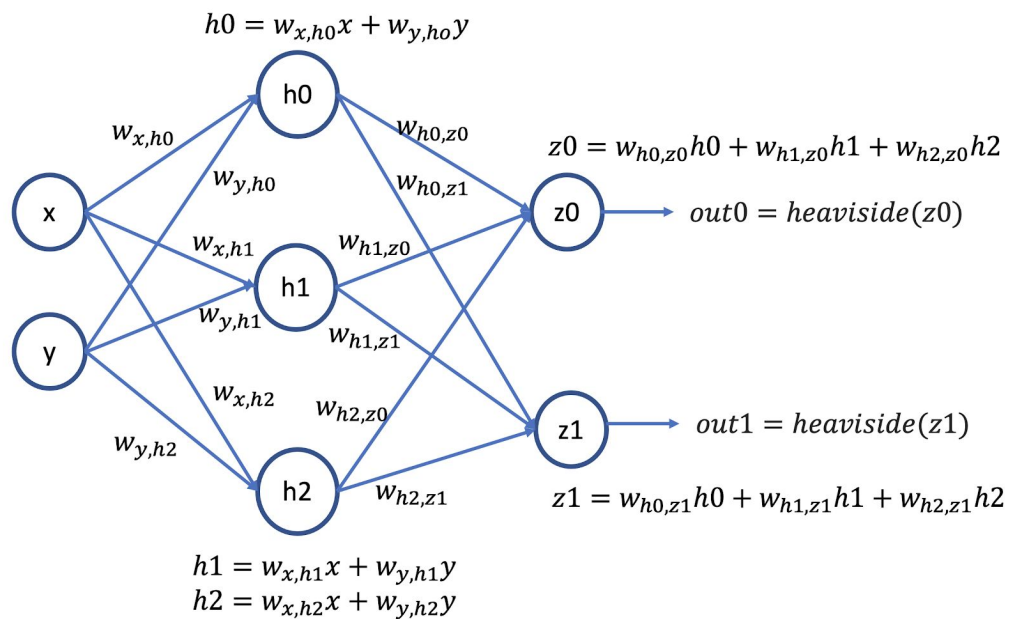
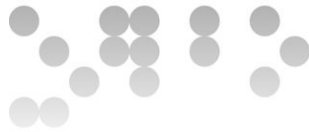
Y0=0: Prob(on) = 1 y Prob(off)=0

Y0=1: Prob(on) = 0 y Prob(off)=1



Apartado 2 (puntos 2)

En lugar de usar directamente las características x e y , obtendremos unas nuevas características $out0$ y $out1$ a partir de una red neuronal. Así pues, en primer lugar, se tienen que calcular la salidas $out0$ y $out1$ de cada una de las instancias (las 16 instancias de la tabla anterior) usando la red neuronal de abajo con los siguientes pesos:



$$\begin{array}{llll}
 w_{x,h0} = 0 & w_{y,h0} = -1 & w_{h0,z0} = 0 & w_{h1,z0} = -1 & w_{h2,z0} = 0 \\
 w_{x,h1} = -1 & w_{y,h1} = 0 & w_{h0,z1} = -1 & w_{h1,z1} = 0 & w_{h2,z1} = -1 \\
 w_{x,h2} = 0 & w_{y,h2} = -1 & & &
 \end{array}$$

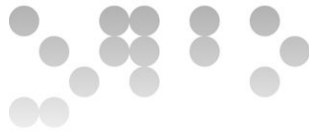
y la siguiente función heaviside que depende del valor de entrada.

$$heaviside(x) = \begin{cases} 0, & \text{if } x \leq 0 \\ 1, & \text{if } x > 0 \end{cases}$$

Solución:

Para cada salida out0 y out1, se coge cada instancia de entrada y se calcula las variables latentes, luego los valores previos a la salida z, y finalmente se aplica la función heaviside para obtener los variables out.

Inst	Clase	X	Y	out0	out1
1	off	-1	2	0	1
2	off	-2	2	0	1
3	on	2	2	1	1
4	on	1	2	1	1
5	on	-3	-1	0	0



6	on	-5	-1	0	0
7	off	4	-1	1	0
8	off	2	-1	1	0
9	off	-2	1	0	1
10	off	-4	1	0	1
11	on	4	4	1	1
12	on	6	7	1	1
13	on	-9	-9	0	0
14	on	-2	-1	0	0
15	off	3	-3	1	0
16	on	-2	1	0	1

Apartado 3 (puntos 2)

Una vez calculados los valores para todas las instancias, debemos construir un árbol de decisión a partir de las salidas de este conjunto de datos calculando la bondad de las particiones de las dos salidas *out0* y *out1*. Así pues, en lugar de entrenar un árbol de decisión sobre las características *x* e *y*, lo haremos sobre las características *out0* y *out1*.

Solución:

Cálculo Bondad

out0:

- *out0*=1 off 3 on 4
- *out0*=0 off 4 on 5
- Bondad: $(5+4)/16 = 9/16$

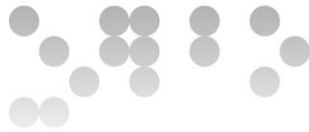
out1:

- *out1*=1 off 4 on 5
- *out1*=0 off 3 on 4
- Bondad: $(5+4)/16 = 9/16$

Al tener todas las características la misma bondad se puede usar cualquiera para dividir los datos.

Cogemos *out0* para dividir los datos aunque se podría dividir los datos con *out1*.

Nodo Raiz *out0*:



- out0=0

Inst	Clase	X	Y	out0	out1
1	off	-1	2	0	1
2	off	-2	2	0	1
5	on	-3	-1	0	0
6	on	-5	-1	0	0
9	off	-2	1	0	1
10	off	-4	1	0	1
13	on	-9	-9	0	0
14	on	-2	-1	0	0
16	on	-2	1	0	1

Ahora calcularemos la bondad con el atributos que falta:

out1:

- out1=1 off 4 on 1
- out1=0 off 0 on 4
- Bondad: $(4+4)/9 = 8/9$

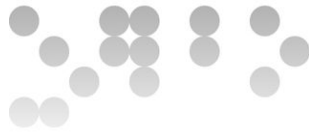
Las instancias restantes se dividen con out1. Siendo la salida de las hojas la siguiente:

out1=0: Prob(on) = 1 y Prob(off)=0

out1=1: Prob(on) = 0.2 y Prob(off)=0.8

- out0=1

Inst	Clase	X	Y	out0	out1
3	on	2	2	1	1
4	on	1	2	1	1
7	off	4	-1	1	0
8	off	2	-1	1	0
11	on	4	4	1	1
12	on	6	7	1	1



15	off	3	-3	1	0
----	-----	---	----	---	---

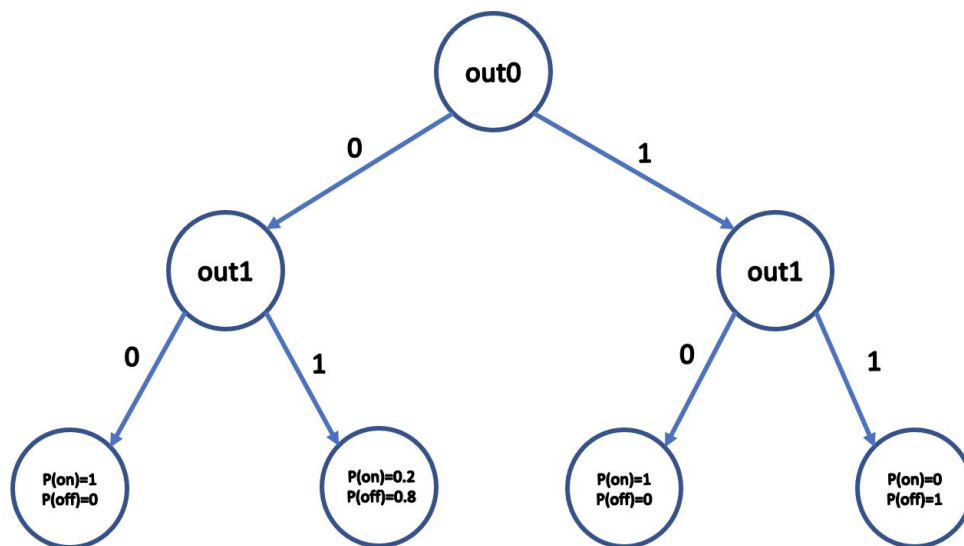
out1:

- out1=1 off 0 on 4
- out1=0 off 3 on 0
- Bondad: $(4+3)/7 = 1$

Las instancias restantes se pueden dividir con out1, siendo la salida de las hojas la siguiente:

out1=0: Prob(on) = 1 y Prob(off)=0

out1=1: Prob(on) = 0 y Prob(off)=1



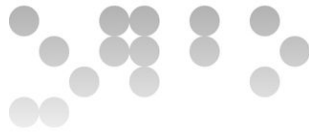
Apartado 4 (puntos 1)

Utilizando el notebook de Python adjunto se debe completar el código para poder entrenar un MultiLayer Perceptron (MLP) con una única salida. Incluir la visualización dada por la función `plt.contourf()` e interpretar los resultados obtenidos de esta visualización.

Solución:

Solamente era necesario crear el clasificador con los parámetros por defecto, aunque en la solución se añade el número máximo de iteraciones y un random state para tener los mismos valores. Finalmente, se entrena el clasificador con los valores de X e y.

```
clf_mlp = MLPClassifier(random_state=1, max_iter=300)
clf_mlp.fit(X,y)
```



Pregunta 2) (2 puntos)

Utilizando el notebook de Python adjunto divide la base de datos para una validación simple para comparar el KNearestNeighbors classifier con todos los datos o reduciendo el número de datos de entrada usando Kmeans. Implementar las partes necesarias para saber el comportamiento del algoritmo del Kmeans y el número de puntos cercanos. ¿Cómo afecta el nombre de clusters utilizados por K-means y el número de vecinos cercanos considerados por el clasificador KNN?

Solución:

El primer paso a implementar era la división de datos en training y test usando la función `train_test_split` indicando un porcentaje necesario para el entrenamiento.

```
# implement simple validation
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=1-percentage_training, random_state=42)
```

Una vez tenemos los datos de entrenamiento, el objetivo era entrenar un `KNeighborsClassifier` como se indica a continuación:

```
#implement kNeighbors classifier
neigh = KNeighborsClassifier(n_neighbors=3)
neigh.fit(X_train, y_train)
```

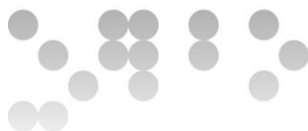
Para reducir el número de elementos del conjunto de train se procede hacer un clustering usando `KMeans` y predecir en que cluster pertenece.

```
kmeans = KMeans(n_clusters=n_clusters, random_state=0).fit(X_train)

kmeans = KMeans(n_clusters=i_cluster, random_state=0).fit(X_train)
y_pred_train = kmeans.fit_predict(X_train)
```

Finalmente, el objetivo era entrenar el `KNeighborsClassifier` usando los clusters aprendidos y el valor del cluster en vez de usar todas las instancias.

```
neigh = KNeighborsClassifier(n_neighbors=i_nn)
neigh.fit(X_train_kmeans, y_train_kmeans)
y_pred = neigh.predict(X_test)
```



Recursos

Para hacer esta PEC el material imprescindible es el módulo 5 y para ejecutar los notebooks en Python del ejercicio recomiendo usar Google Colab que es gratuito (<https://colab.research.google.com>).

Criterios de valoración

Las puntuaciones se muestran en cada pregunta del enunciado.

Formato y fecha de entrega

Para dudas y aclaraciones sobre el enunciado, dirigiros al consultor responsable del aula.

Hay que entregar la solución en un archivo PDF usando una de las plantillas entregadas conjuntamente con este enunciado. Adjuntar el fichero a un mensaje en el apartado Entrega y Registro de EC (REC).

El nombre del archivo debe ser *Apellidos_Nombre_IA_PEC4* con la extensión .pdf (PDF).

La fecha límite de entrega es el: **20/12/2020** (a las 24 horas).

Razonad la respuesta en todos los ejercicios. Las respuestas sin justificación no recibirán puntuación.

Nota: Propiedad intelectual

A menudo es inevitable, al producir una obra multimedia, hacer uso de recursos creados por terceras personas. Es por tanto comprensible hacerlo en el marco de una práctica de los estudios de Informática, siempre que se documente claramente y no suponga plagio en la práctica.

Por lo tanto, al presentar una práctica que haga uso de recursos ajenos, se presentará junto con ella un documento en el que se detallen todos ellos, especificando el nombre de cada recurso, su autor, el lugar donde se obtuvo y el su estatus legal: si la obra está protegida por copyright o se acoge a alguna otra licencia de uso (Creative Commons, licencia GNU, GPL ...).

El estudiante deberá asegurarse de que la licencia que sea no impide específicamente su uso en el marco de la práctica. En caso de no encontrar la información correspondiente deberá asumir que la obra está protegida por copyright.

Deberán, además, adjuntar los archivos originales cuando las obras utilizadas sean digitales, y su código fuente esté correspondiente.